

Como AutoML e Algoritmos Genéticos Podem Otimizar Redes CNNs

Humberto Tavora
Centro de Informática
Universidade Federal de Pernambuco
Recife, Brazil
ORCID 0009-0001-0713-6408

Caio Rocha
Centro de Informática
Universidade Federal de Pernambuco
Recife, Brazil
crc@cin.ufpe.br

Resumo—A ascensão do AutoML nos últimos anos reflete a crescente demanda por automação nas tarefas demoradas e iterativas associadas ao desenvolvimento de modelos de aprendizado de máquina (ML). O objetivo central do AutoML é reduzir a dependência de especialistas na criação de modelos preditivos, aumentando, assim, a produtividade e impulsionando o avanço da pesquisa em deep learning. Este trabalho propõe uma metodologia de AutoML implementada através de algoritmos genéticos para otimizar automaticamente a busca por arquiteturas de CNNs.

Palavras-chave—autoML, algoritmos genéticos, automação, CNN

I. INTRODUÇÃO

O uso de modelos de aprendizado de máquina se elevou como nunca visto nos últimos anos, onde se demonstrou capaz de fazer parte de diversas atividades de rotina. No entanto, o desenvolvimento desses modelos ainda é bastante custoso, sobretudo no que diz respeito ao custo de treinamento, já que existe bastante trabalho de tentativa e erro quando falamos da definição dos modelos. Nesse contexto, a implementação de tecnologias de AutoML (Aprendizado de Máquina Automatizado) surge como uma solução promissora, automatizando as etapas do pipeline de desenvolvimento tradicional e aumentando a eficiência ao reduzir erros humanos. Com essa importância, o campo é bastante estudado, evidenciado pela intensificação dos esforços na indústria e pelo volume de publicações acadêmicas dedicadas ao tema. No entanto, apesar dos avanços significativos, ainda existem desafios a serem enfrentados [1].

Embora o AutoML possa otimizar modelos para datasets específicos, a generalização destes para diferentes domínios e problemas ainda é um desafio. Além disso, como visto em [2], modelos de aprendizado de máquina frequentemente são considerados "caixas-pretas", o que dificulta a interpretação de suas decisões; a automatização do aprendizado de máquina adiciona uma camada a mais na dificuldade de transparência do modelo. Melhorar a explicabilidade dos modelos gerados por AutoML é crucial, especialmente em áreas como saúde e finanças, onde as decisões precisam ser compreensíveis e justificáveis para os usuários finais. Além disso, apesar dos avanços, a otimização de hiperparâmetros ainda pode ser um processo demorado e dependente de experiências anteriores

[3], [4]. Novas técnicas de otimização, incluindo métodos que considerem interações complexas entre hiperparâmetros, precisam ser exploradas para aumentar a eficiência e a eficácia do processo de ajuste.

Nesse contexto, diferentes implementações de otimização de hiperparâmetros de AutoML já foram propostas. Um tipo de implementação em particular se destaca pela eficiência em estudos recentes, como visto em [5], [6] e [4]. Trata-se da utilização de algoritmos genéticos. Este estudo propõe a utilização de algoritmos genéticos (GA) na busca automática pela arquitetura de redes neurais convolucionais, contribuindo para o campo ao discutir as vantagens da inclusão de GA em comparação com métodos tradicionais, tanto em questão de performance como de complexidade e esforço.

II. REVISÃO DA LITERATURA

A. CNN

Convolutional Neural Networks (CNNs) emergem como uma abordagem promissora no campo do aprendizado profundo, destacando-se por sua capacidade de reconhecimento espacial da imagem e extração de características. Desde seus primeiros artigos relevantes [7] demonstra excelente desempenho de aprendizado em uma ampla gama de aplicações reais, incluindo reconhecimento de imagens [8], fala e tarefas de reforço [9] e reconhecimento de expressões faciais [10]. Sua arquitetura é composta por camadas convolucionais, de pooling e totalmente conectadas, o que permite a extração hierárquica de características e uma modelagem robusta dos dados. Esse design tem sido a base de arquiteturas de sucesso como GoogleNet [11], ResNet [12] e DenseNet [13], que, embora eficazes, dependem de decisões criteriosas sobre inúmeros hiperparâmetros e configurações estruturais, como profundidade da rede, tipos de camadas e conectividade.

O projeto manual de CNNs apresenta desafios significativos. Apesar de seu poder de generalização, o desempenho das CNNs depende fortemente da escolha de sua configuração ou hiperparâmetros. Determinar a profundidade da rede (número de camadas convolucionais e totalmente conectadas) e o número de filtros em cada camada são escolhas fundamentais. Redes muito rasas podem ter baixa capacidade de generalização, enquanto redes muito profundas podem causar overfitting e aumentar o tempo de processamento. O tamanho

dos filtros define a resolução espacial das características extraídas. Filtros pequenos (3x3) são comuns e permitem uma extração detalhada e eficiente, mas o tamanho ideal pode variar conforme o tipo de imagem e o nível de detalhe necessário. Embora a ReLU seja amplamente utilizada, outras funções, como Leaky ReLU ou Swish, podem melhorar a performance dependendo das características do problema e do modelo. Por fim, a escolha do otimizador e da função de perda impacta diretamente a eficiência do treinamento. Para redes profundas, otimizadores como Adam ou RMSprop podem facilitar a convergência.

De maneira geral, o projeto de CNNs requer conhecimentos especializados tanto no domínio dos dados quanto na construção de redes neurais, uma combinação frequentemente ausente entre os usuários. Este cenário tem incentivado o desenvolvimento de métodos automáticos de projeto de arquiteturas de CNN, que democratizam o acesso a modelos eficientes e permitem a expansão de sua aplicação em cenários variados. Esses métodos podem ser classificados em categorias "automáticas + ajuste manual" e "totalmente automáticas", com diferentes níveis de dependência do conhecimento prévio do usuário [14].

B. AutoML e Algoritmo Genético

O Aprendizado de Máquina Automatizado (AutoML) surge para simplificar o processo de ponta a ponta de desenvolvimento de modelos de aprendizado de máquina. Ao automatizar tarefas como feature engineering, seleção de modelos, ajuste de hiperparâmetros e otimização de pipeline, o AutoML capacita cientistas de dados e especialistas em domínio a se concentrar em considerações estratégicas de alto nível, ao invés das complexidades técnicas de baixo nível, além de diminuir a barreira para que os não especialistas possam utilizar essas ferramentas com mais facilidade.

Aliado a isso, o Algoritmo Genético (GA) é uma técnica versátil de otimização amplamente utilizada no campo de aprendizado de máquina [15], [16]. Ao imitar os princípios da seleção natural, o GA evolui uma população de soluções potenciais ao longo de gerações sucessivas, melhorando iterativamente a sua adequação. Um exemplo de implementação pode ser visto em Fig. 1.

Como já abordado, a seleção adequada de hiperparâmetros em CNNs é complexa e depende de muita experimentação, exigindo um trabalho de tentativa e erro bastante custoso. Por isso, técnicas de otimização automática têm surgido, entre elas, implementações que utilizam como base técnicas de algoritmos genéticos [6], [4]. A integração de algoritmos genéticos e estratégias de busca automática permite explorar grandes espaços de arquitetura de maneira eficiente, promovendo a inovação e a generalização em projetos de aprendizado profundo.

No estudo da utilização de métodos evolutivos para esse tipo de automação, técnicas como o CGP (Cartesian Genetic Programming) aplicado a CNNs (CGP-CNN) [17], e os métodos de aprendizado por reforço, como NASNet [18], se demonstraram bastante promissoras. Apesar de os métodos baseados

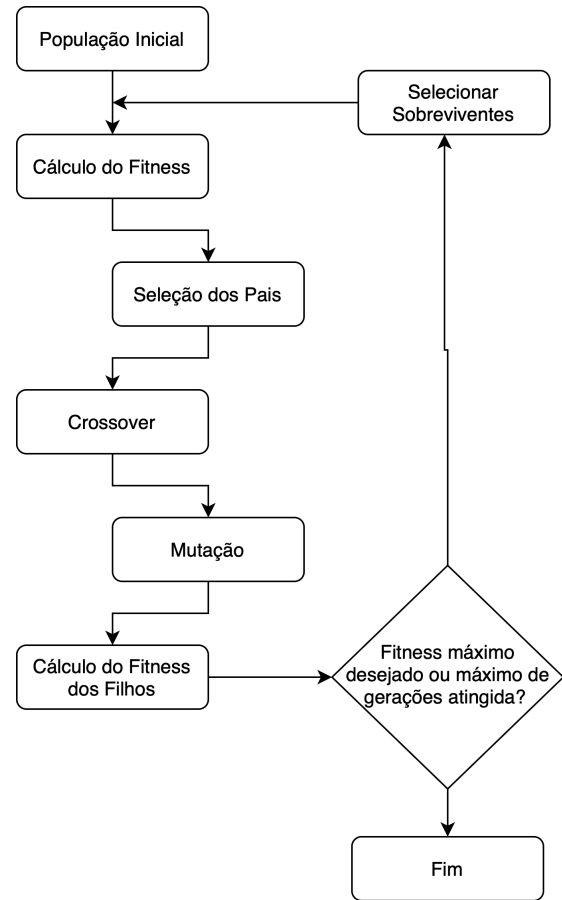


Figura 1: Fluxograma de uma implementação de um algoritmo genético

em aprendizado por reforço frequentemente exibirem alta performance, eles demandam recursos computacionais significativamente maiores em comparação aos métodos baseados em algoritmos evolutivos [19]. Por exemplo, o NASNet consumiu 800 GPUs durante 28 dias para otimizar uma arquitetura na base CIFAR-10, enquanto a abordagem Genetic CNN alcançou desempenho comparável com apenas 17 GPUs em um único dia.

Assim, avaliar e otimizar CNNs por meio de abordagens automáticas alinha-se diretamente com as demandas contemporâneas de escalabilidade e acessibilidade em inteligência artificial.

III. METODOLOGIA

A. Implementação

O algoritmo desenvolvido seguiu como base uma implementação básica de algoritmos genéticos semelhante ao mostrado em Fig. 1 e seu pseudocódigo pode ser visto em Algo. 1:

O algoritmo desenvolvido inicia com uma população de N indivíduos. Sobre ela, é feito o treinamento por E épocas, e por fim, realizada a etapa de testes. Neste momento, é atribuído o valor de fitness do indivíduo, que utiliza como

Algorithm 1 Evolução do Algoritmo Genético para CNNs

Require: Tamanho da população N , Taxa de mutação p_{mut} ,

Gerações Máximas G , Fitness alvo f_{goal} , Épocas E

```
1: População Inicial  $\mathcal{P} = \{\mathbf{N}_i : \mathbf{N}_i \sim \mathcal{CNN}(DNA)\}_{i=1}^N$ 
2: Treinar( $E$ )  $\mathbf{N}_i$  e calcular  $f(\mathbf{N}_i)$  (Incluir na Cache)
3: for  $g = 1, 2, \dots, G$  do
4:   for  $\mathbf{N}_i \in \mathcal{P}$  do
5:     if  $f(\mathbf{N}_i) > f_{best}$  then
6:        $f_{best} \leftarrow f(\mathbf{N}_i)$ ,  $\mathbf{N}_{best} \leftarrow \mathbf{N}_i$ 
7:     end if
8:   end for
9:   if  $f_{best} \geq f_{goal}$  then
10:    Stop
11:  end if
12:  Selecionar os Pais  $\mathbf{N}_{p1}, \mathbf{N}_{p2} \in \mathcal{P}$  com probabilidade
     $\propto f(\mathbf{N}_i)$ 
13:   $\mathbf{c}_1, \mathbf{c}_2 \leftarrow \text{Crossover}(\mathbf{N}_{p1}, \mathbf{N}_{p2})$ 
14:   $\mathbf{c}_1 \leftarrow \text{Muta\c{c}o\~{e}}(\mathbf{c}_1, p_{mut})$ ,  $\mathbf{c}_2 \leftarrow \text{Muta\c{c}o\~{e}}(\mathbf{c}_2, p_{mut})$ 
15:   $\mathcal{P} \leftarrow \mathcal{P} \cup \{\mathbf{c}_1, \mathbf{c}_2\}$ 
16:  for  $\mathbf{N}_i \in \mathcal{P}$  do
17:    if  $\mathbf{N}_i$  not in Cache then
18:      Treinar( $E$ )  $\mathbf{N}_i$  e calcular  $f(\mathbf{N}_i)$  (Incluir na Cache)
19:    end if
20:  end for
21:  Selecionar os Sobreviventes ( $N$  indivíduos de maior
     $f(\mathbf{N}_i)$  em  $\mathcal{P}$ )
22: end for
23: Return  $\mathbf{N}_{best}, f_{best}$ 
```

base informações obtidas na etapa de testes. Então, durante G gerações (ou até ser encontrado o valor de fitness algo f_{goal}) será simulado um ambiente evolutivo, onde, em cada geração, teremos a seleção de pais, com base em **Seleção por Roleta Viciada** [20].

Esses pais selecionados serão utilizados para gerar 2 novos indivíduos. Esses, são os novos filhos que resultam das operações de crossover e mutação dos pais. Na etapa de crossover, será feita a combinação genética dos pais para criar um descendente, simulando o processo de reprodução sexual, misturando as características na esperança de produzir descendentes melhores. Já na etapa de mutação, com base em p_{mut} , é alterado um ou mais genes dos filhos com base nos hiperparâmetros da população inicial. Essa etapa simula erros de replicação genética e tem como objetivo introduzir variabilidade na população, almejando evitar que o algoritmo fique preso em ótimos locais.

Por fim, é realizada a etapa de seleção de sobreviventes, onde mais uma vez com base na seleção por roleta viciada, é definido quem irá para a próxima geração. É importante notar que a cada cálculo de fitness é utilizada a *Cache*. Semelhante à estratégia tomada em [14], esta estrutura serve para garantir que apenas os indivíduos (CNNs) que não tiveram o seu fitness já calculado, passaram pela etapa de treinamento e teste. Desta forma, garantimos que todos os indivíduos passam pela mesma

quantidade de treinamento (E épocas) e reduzimos o custo computacional consideravelmente. Além disso, em cada etapa de treinamento e teste, é dada prioridade para a utilização da GPU, o que permite explorar o paralelismo deste tipo de computação.

A implementação completa pode ser vista em section VIII

B. Bases de dados

Afim de validar as arquiteturas evoluídas pela implementação apresentada, foram escolhidas as seguintes bases de dados. A escolha foi motivada pela necessidade de avaliar a generalização e a aplicabilidade do método proposto em diferentes domínios, indo de cenários simples e controlados a problemas complexos e de alta relevância prática.

1) *MINIST*: A base de dados MNIST é um conjunto amplamente utilizado para benchmarking em tarefas de classificação de imagens. Ela contém 70.000 imagens em tons de cinza de dígitos manuscritos (de 0 a 9) com resolução de 28x28 pixels. É frequentemente utilizada como referência em estudos de aprendizado de máquina devido à sua simplicidade e escalabilidade.

2) *Chest X-Ray Images (Pneumonia)*: Essa base de dados contém imagens de raios-X do tórax categorizadas em duas classes: normal e pneumonia. Ela é composta por mais de 5.000 imagens, sendo utilizada para tarefas de classificação médica e diagnóstico assistido por IA, com o objetivo de identificar pneumonia a partir de radiografias.

3) *COVID-19 Radiography Database*: Esse conjunto de dados foi projetado para apoiar pesquisas no diagnóstico de Covid-19. Ele contém imagens de raios-X organizadas em três classes: normais, pneumonia e Covid-19. A base possui mais de 21.000 imagens, fornecendo um recurso robusto para estudos relacionados à detecção de Covid-19 e doenças respiratórias correlacionadas.

C. Cálculo de Fitness

Em trabalhos semelhantes como vistos em [5], [6] e [4] é comum que o fitness atribuído aos indivíduos seja considerado como sendo a acurácia obtida na etapa de testes. No entanto, a utilização da acurácia como métrica de avaliação em modelos de aprendizado de máquina apresenta limitações significativas que podem comprometer a interpretação dos resultados obtidos [21]. Essa métrica, ao considerar apenas a proporção de previsões corretas em relação ao total, trata todos os erros de maneira uniforme, ignorando a relevância contextual de falsos positivos e falsos negativos [22]. Em muitos cenários, como na detecção de doenças ou em sistemas de segurança, os custos associados a diferentes tipos de erro não são equivalentes. Por exemplo, um falso negativo na detecção de pneumonia pode resultar em consequências graves para a saúde do paciente, enquanto um falso positivo, embora indesejado, pode ser menos prejudicial.

Além disso, a acurácia assume implicitamente que todos os modos de falha possuem o mesmo impacto, o que pode não refletir a realidade de problemas práticos, especialmente

em conjuntos de dados desbalanceados. Quando a classe majoritária domina o conjunto de dados, a acurácia pode ser artificialmente inflada, mascarando a incapacidade do modelo de identificar corretamente instâncias pertencentes à classe minoritária. Assim, a utilização exclusiva dessa métrica pode obscurecer insights críticos sobre o desempenho do modelo, impedindo uma análise mais detalhada e fundamentada.

Dessa forma, para uma avaliação mais robusta e abrangente, recomenda-se a utilização de métricas complementares, como a precisão (precision), a sensibilidade (recall) e a métrica F1, que permitem captar nuances relacionadas ao desempenho do modelo em diferentes tipos de erro. Em cenários específicos, métricas personalizadas, ajustadas às necessidades e aos custos associados a cada tipo de erro, podem oferecer uma análise mais significativa e alinhada com os objetivos do sistema em desenvolvimento. Neste trabalho, utilizaremos uma relação entre o F1-Score e a acurácia. Desta forma, é possível capturar tanto a capacidade geral de predição correta quanto a sensibilidade aos diferentes tipos de erro. Essa relação será definida como:

$$F = \alpha \cdot A + (1 - \alpha) \cdot F1$$

O alpha permite ajuste flexível de acordo com as prioridades do problema.

IV. RESULTADOS

Para as simulações, foram utilizadas várias combinações de hiperparâmetros. Inicialmente, apenas a profundidade, quantidade de filtros, função de ativação e otimizador foram utilizados. Para as primeiras simulações, era suficiente, no entanto, para problemas mais complexos, existiam técnicas a serem exploradas a fim de extrair mais performance dos modelos, como, por exemplo, técnicas de inicialização. Além disso, o tamanho do Kernel foi um ponto que precisou ser levado em consideração, já que, para problemas mais complexos, uma extração de características mais detalhada é necessária. Ao final, foi necessária uma grade de hiperparâmetros como a seguinte I:

Tabela I: Grade de Hiperparâmetros

Hiperparâmetro	Valores
Profundidade da Rede (<i>depth</i>)	{1, 2, 3}
Tamanho do Kernel	{(3, 3), (5, 5), (7, 7)}
Filtros por Camada	{32, 64, 128}
Funções de Ativação	{ <i>relu</i> , <i>tanh</i> , <i>sigmoid</i> }
Otimizadores	{ <i>sgd</i> , <i>adam</i> }
Funções de Perda	{ <i>cat_cross</i> , <i>binary_cross</i> , <i>poisson</i> }
Inicializadores de Pesos	{ <i>he_normal</i> , <i>glorot_uniform</i> , <i>None</i> }

A. Simulações

Foram realizadas diversas simulações, evoluindo as redes até um fitness alvo ou até o limite de gerações definido. O tempo necessário para cada simulação depende do problema alvo e da base de dados utilizada. Por exemplo, para a base MNIST, a evolução leva cerca de 6 minutos (para 15 gerações com 5 indivíduos e 2 filhos por geração) utilizando

a CPU especificada em section VIII, enquanto as outras bases mencionadas em subsection III-B podem levar mais de 4 horas. Por isso, na implementação do código é sempre levado como prioridade a utilização da GPU, o que em nossos testes tende a acelerar a evolução em pelo menos 10 vezes (para a CPU especificada em section VIII). Também é recomendado que o fitness alvo seja escolhido com cuidado, levando em consideração a complexidade da base. Por exemplo, para a base MNIST é razoável exigir um fitness de pelo menos 98%, já para as demais, podemos começar com um fitness menor por volta de 85% e ir aumentando no decorrer dos testes.

As evoluções a seguir foram realizadas levando em consideração um total de 15 gerações como máximo permitido e 5 épocas por rede. Essa quantidade é pequena o suficiente para não trazer problemas de performance em bases mais complexas, mas alta o suficiente para demonstrar a trajetória do fitness médio e máximo em cada evolução. A taxa de mutação foi mantida em 15%, e o fitness alvo foi variando a depender da base utilizada, como já foi mencionado. Também é importante mencionar que a grade de hiperparâmetros impacta bastante na performance da evolução, já que, se permitirmos uma rede com profundidade alta demais, o custo para o seu treinamento vai atuar como um overhead para a evolução. Por isso, a grade em I foi tomada como base e ajustada para a complexidade do problema.

V. EVOLUÇÕES

As primeiras evoluções foram feitas para a base MNIST. Apesar de uma base bastante simples, podemos observar o comportamento da evolução ao longo das gerações e ter uma estimativa de performance e ajustes que seriam necessários no código. Como podemos observar em 3, pela simplicidade da base escolhida, os primeiros indivíduos já possuíam um fitness muito alto (97,7%), por isso, a evolução demora a encontrar um fitness melhor, e quando o faz, ele já é o fitness final já que o nosso alvo é 98%. É interessante observar que o fitness médio em 2 variou bastante ao longo da evolução. Isso nos diz que existiam indivíduos na população ao longo das gerações com um fitness menor mas que sobreviveram.

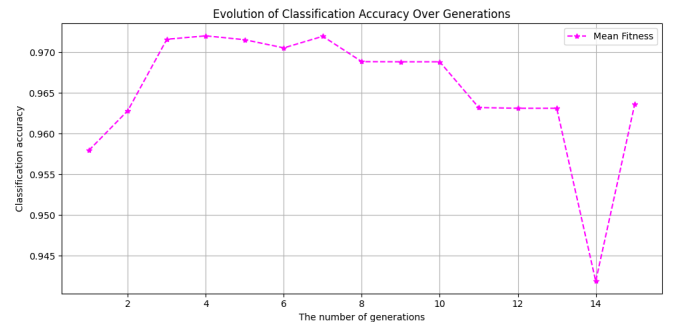


Figura 2: Trajetória do Fitness médio para Base MNIST - 1

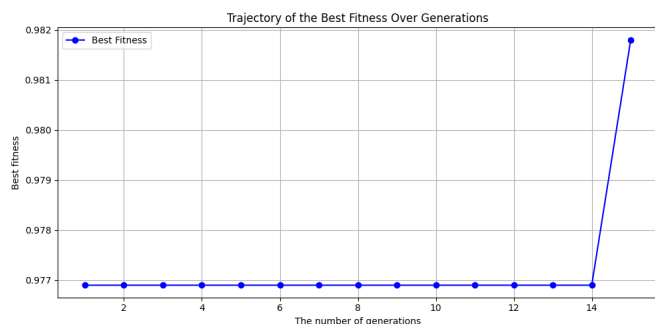


Figura 3: Trajetória do melhor Fitness para Base MNIST - 1

Isso permite que a evolução explore mais combinações e tenha a capacidade de explorar soluções menos óbvias, o que tende a escapar de mínimos locais. Essa estratégia, inclusive, foi a responsável por encontrar o fitness máximo, onde na geração 14, o fitness médio cai, e na geração seguinte sobe de volta e encontra um fitness de aproximadamente 98,2%. Essa estratégia se confirmou ao se analisar os logs de saída da evolução.

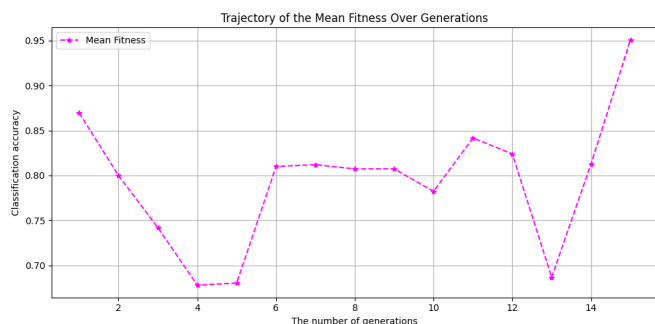


Figura 4: Trajetória do Fitness médio para Base MNIST - 2

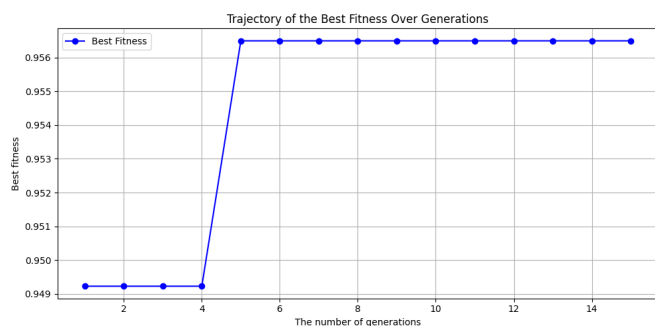


Figura 5: Trajetória do melhor Fitness para Base MNIST - 2

Em outro cenário, como visto em Fig.5 e Fig.4, é interessante notar que o melhor fitness encontrado pela evolução foi de 95,6%, e a geração chega ao fim por conta do limite de gerações máximas permitidas. É razoável esperar que, se aumentarmos a quantidade de gerações permitidas, a evolução encontraria uma rede com o fitness mínimo esperado, principalmente se observarmos que o fitness médio, na geração 15,

fica muito próximo ao melhor fitness, indicando que todos os indivíduos daquela geração são boas soluções.

Para bases mais complexas como a Chest X-Ray Images (Pneumonia), o resultado foi bastante promissor. A evolução foi capaz de encontrar uma solução com fitness de 90%, com acurácia e F1-score muito próximos um do outro. Essa performance é equivalente às melhores soluções da plataforma de competição Kaggle, e com a vantagem de apresentar um F1-Score alto, o que é bastante relevante no contexto médico.

Nas trajetórias de média Fig.6 e melhor fitness Fig.7 podemos observar que a evolução encontrou uma solução boa já no início e deu passos consistentes até o melhor fitness ao final da evolução. Estes resultados não incluem técnicas mais avançadas como Dropout e diferentes abordagens de regularização, o que é comum em modelos de performance semelhante. É esperado que incluir essas técnicas no algoritmo permita que a evolução explore ainda melhor as novas soluções promissoras. Importante mencionar que foi necessário aumentar a quantidade máxima de gerações permitidas na evolução, já que o problema é mais complexo e precisa explorar mais soluções (O mesmo se aplica para a base de COVID).

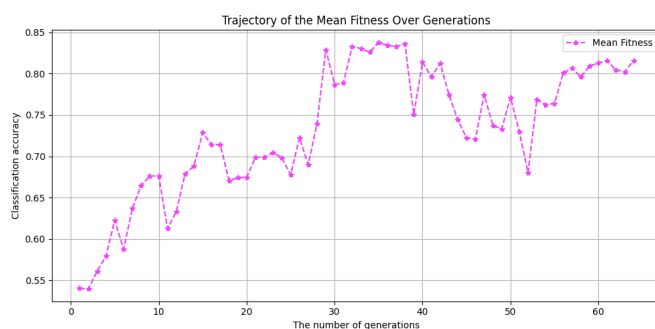


Figura 6: Trajetória do Fitness médio para Base Chest X-Ray

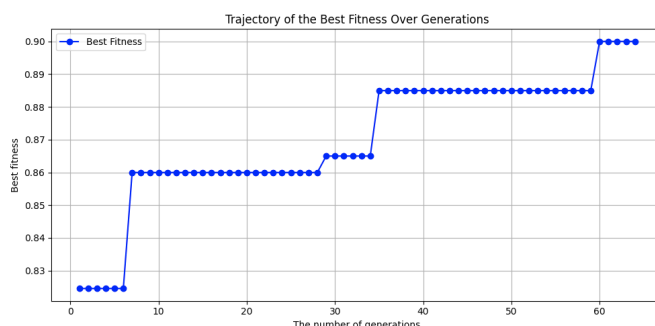


Figura 7: Trajetória do melhor Fitness para Base Chest X-Ray

Para a base de COVID, podemos observar que o resultado foi excelente, apresentando um fitness de mais de 95%. Durante a trajetória do fitness Fig.9, pode-se observar que a primeira geração já encontrou uma ótima solução, e com isso, apenas um salto de fitness foi feito para o resultado final. Ao observar a trajetória do fitness médio Fig.8, podemos observar que o fitness dos indivíduos sobreviventes foi consistentemente

melhorando ao longo do tempo, e isso permitiu o salto do melhor fitness acontecer.

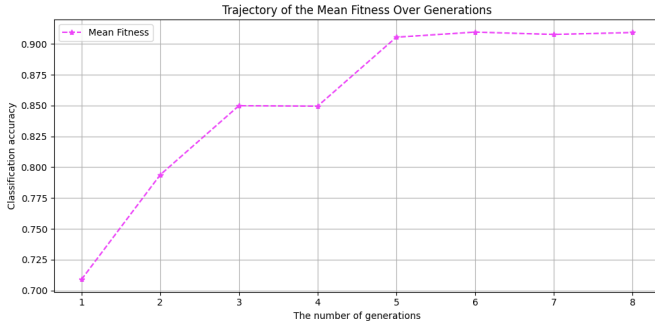


Figura 8: Trajetória do Fitness médio para Base COVID

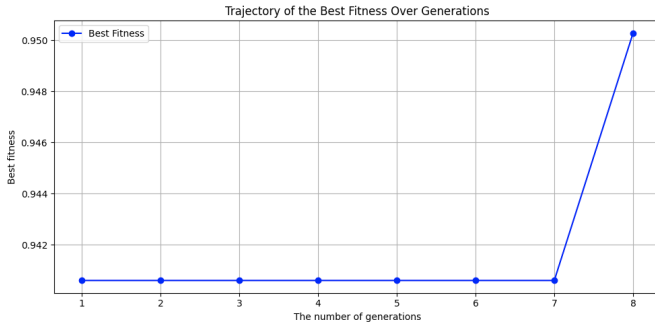


Figura 9: Trajetória do melhor Fitness para Base COVID

VI. COMPARAÇÃO COM O ESTADO DA ARTE

Dentre as ferramentas de otimização automática de modelos de machine learning, a principal utilizada atualmente é a Optuna [23]. O Optuna utiliza técnicas como a otimização bayesiana e busca aleatória para explorar o espaço de hiperparâmetros. Ele ajusta seus experimentos com base em modelos probabilísticos, prevendo regiões promissoras no espaço de busca. Após cada avaliação, ele atualiza suas crenças sobre onde estão os melhores resultados (através de modelos como Gaussian Processes ou Tree-structured Parzen Estimators).

O Optuna explora rapidamente o espaço inicial, mas tende a concentrar-se em áreas promissoras, possivelmente ignorando configurações fora desse foco. É ideal para espaços de busca relativamente pequenos ou para problemas com recursos computacionais limitados. Além disso, usa a informação dos resultados anteriores para direcionar futuras buscas de maneira inteligente, ajustando a estratégia de busca conforme os resultados acumulados, o que o torna eficiente em problemas de baixa dimensionalidade.

A abordagem por Algoritmos Genéticos difere do Optuna fundamentalmente na [24] na estratégia de exploração do espaço de busca. O GA é particularmente eficiente em explorar amplamente o espaço de busca e encontrar configurações em áreas menos óbvias, inclusive comprovamos isso nos experimentos onde o fitness diminui para depois evoluir novamente. Além disso, os cromossomos podem ser avaliados independentemente, tornando o GA adequado para execução

em arquiteturas paralelas ou distribuídas, o que se beneficia bastante do uso de GPUs e linguagem de alta performance e paralelismo como a linguagem Julia [25]. Além disso, a abordagem por Algoritmos Genéticos é mais adequada para otimizar redes neurais com muitos hiperparâmetros interdependentes (como o número de camadas, tamanho de filtros, taxa de aprendizado, etc.). Em cenários onde o espaço de busca não é contínuo ou possui dependências complexas, o GA explora soluções de maneira mais eficiente.

Nos nossos testes, utilizando Optuna fomos capazes de encontrar a melhor combinação para o MNIST em 8 minutos, com a configuração tendo 98,4% de acurácia. Na abordagem por GA, o fitness final foi de 98,2%, o que já é bom o suficiente para grande parte dos problemas, mas além disso, pudemos garantir que o F1-score também fosse levado em consideração durante a otimização, o que é relevante para problemas mais críticos. Além disso, a abordagem genética levou apenas 3 minutos. Essa diferença de tempo tende a aumentar em problemas de maior complexidade, já que a abordagem por GA se destaca em cenários de alta dimensionalidade.

VII. CONCLUSÃO

Com base nos resultados obtidos, é possível concluir que o algoritmo genético proposto demonstrou ser uma abordagem eficaz para a evolução de redes neurais convolucionais (CNNs), permitindo a otimização de hiperparâmetros em diferentes domínios. A utilização de estratégias como roleta viciada para a seleção de indivíduos, o uso de crossover e mutação para promover variabilidade, e a implementação de um cache para otimizar o cálculo de fitness contribuíram para o bom desempenho do método.

Os experimentos realizados com as bases de dados MNIST, Chest X-Ray Images (Pneumonia) e COVID-19 Radiography Database evidenciaram a capacidade do algoritmo em encontrar soluções adequadas tanto para problemas simples quanto para cenários complexos. Para a base MNIST, os resultados mostraram que o algoritmo é capaz de alcançar altos valores de fitness rapidamente, com trajetórias de evolução que demonstram exploração eficaz do espaço de soluções. Por outro lado, em bases mais complexas, como Chest X-Ray e COVID-19 Radiography, os resultados foram consistentes com soluções de referência, destacando a robustez do método e a relevância de métricas como o F1-Score em cenários médicos.

A abordagem também evidenciou que a definição de parâmetros como a profundidade da rede, o tamanho do kernel e os inicializadores de pesos impacta significativamente a performance da evolução. Além disso, a priorização do uso de GPU contribuiu para uma aceleração expressiva do processo, especialmente em problemas de maior complexidade.

Por fim, os resultados destacam a importância de utilizar métricas que vão além da acurácia para a avaliação do desempenho, especialmente em problemas desbalanceados ou sensíveis a diferentes tipos de erro, como em aplicações médicas. A combinação de acurácia e F1-Score, ajustada por um fator α , mostrou-se uma escolha adequada para capturar

tanto a capacidade geral de predição quanto a sensibilidade aos diferentes tipos de erro.

Com base nesses achados, a metodologia proposta apresenta potencial significativo para ser aplicada em problemas reais, podendo ser expandida e adaptada para cenários mais complexos ou restrições computacionais específicas. Estudos futuros podem explorar melhorias no algoritmo, como ajustes dinâmicos de hiperparâmetros ou a inclusão de estratégias híbridas com outras abordagens de otimização.

VIII. ONLINE RESOURCES

O algoritmo genético desenvolvido foi publicado e disponibilizado no github, e pode ser acessado pelo repositório Neural Network Architecture and Hyperparameter Optimization.

REFERÊNCIAS

- [1] S. K. Karmaker, M. M. Hassan, M. J. Smith, L. Xu, C. Zhai, and K. Veeramachaneni, "Automl to date and beyond: Challenges and opportunities," *ACM Computing Surveys (CSUR)*, vol. 54, no. 8, pp. 1–36, 2021.
- [2] Y. Sun, Q. Song, X. Gui, F. Ma, and T. Wang, "Automl in the wild: Obstacles, workarounds, and expectations," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023, pp. 1–15.
- [3] M. Feurer and F. Hutter, "Hyperparameter optimization," pp. 3–38.
- [4] C. Spandana, I. V. Srisurya, S. A. Nandhini, R. P. Kumar, G. B. Mohan, and P. Srinivasan, "An efficient genetic algorithm based auto ml approach for classification and regression," in *2023 International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*. IEEE, 2023, pp. 371–376.
- [5] L. Cardoso, V. Santos, R. K. Francês, and R. Alves, "Automl optimized by genetic algorithm and item response theory," in *Anais do XX Encontro Nacional de Inteligência Artificial e Computacional*. Porto Alegre, RS, Brasil: SBC, 2023, pp. 1089–1098. [Online]. Available: <https://sol.sbc.org.br/index.php/eniac/article/view/25766>
- [6] D. Wu, Q. Guan, Z. Fan, H. Deng, and T. Wu, "Automl with parallel genetic algorithm for fast hyperparameters optimization in efficient iot time series prediction," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 9, pp. 9555–9564, 2022.
- [7] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.
- [9] T. N. Sainath, B. Kingsbury, A.-r. Mohamed, G. E. Dahl, G. Saon, H. Soltan, T. Beran, A. Y. Aravkin, and B. Ramabhadran, "Improvements to deep convolutional neural networks for lvcsr," in *2013 IEEE workshop on automatic speech recognition and understanding*. IEEE, 2013, pp. 315–320.
- [10] G. Meena, K. K. Mohbey, A. Indian, M. Z. Khan, and S. Kumar, "Identifying emotions from facial expressions using a deep convolutional neural network-based approach," *Multimedia Tools and Applications*, vol. 83, no. 6, pp. 15 711–15 732, 2024.
- [11] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [13] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.
- [14] Y. Sun, B. Xue, M. Zhang, G. G. Yen, and J. Lv, "Automatically designing cnn architectures using the genetic algorithm for image classification," *IEEE transactions on cybernetics*, vol. 50, no. 9, pp. 3840–3854, 2020.
- [15] S. Ahn, J. Kim, S. Y. Park, and S. Cho, "Explaining deep learning-based traffic classification using a genetic algorithm," *IEEE Access*, vol. 9, pp. 4738–4751, 2020.
- [16] R. M. Aziz, R. Mahto, K. Goel, A. Das, P. Kumar, and A. Saxena, "Modified genetic algorithm with deep learning for fraud transactions of ethereum smart contract," *Applied Sciences*, vol. 13, no. 2, p. 697, 2023.
- [17] M. Suganuma, S. Shirakawa, and T. Nagao, "A genetic programming approach to designing convolutional neural network architectures," in *Proceedings of the genetic and evolutionary computation conference*, 2017, pp. 497–504.
- [18] B. Zoph, "Neural architecture search with reinforcement learning," *arXiv preprint arXiv:1611.01578*, 2016.
- [19] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Evolving deep convolutional neural networks for image classification," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 2, pp. 394–407, 2019.
- [20] A. Lipowski and D. Lipowska, "Roulette-wheel selection via stochastic acceptance," *Physica A: Statistical Mechanics and its Applications*, vol. 391, no. 6, pp. 2193–2196, 2012.
- [21] B. Hutchinson, N. Rostamzadeh, C. Greer, K. Heller, and V. Prabhakaran, "Evaluation gaps in machine learning practice," in *Proceedings of the 2022 ACM conference on fairness, accountability, and transparency*, 2022, pp. 1859–1876.
- [22] M. Ghanem, A. K. Ghaith, V. G. El-Hajj, A. Bhandarkar, A. de Giorgio, A. Elmi-Terander, and M. Bydon, "Limitations in evaluating machine learning models for imbalanced binary outcome classification in spine surgery: a systematic review," *Brain Sciences*, vol. 13, no. 12, p. 1723, 2023.
- [23] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2623–2631.
- [24] T. Agrawal and T. Agrawal, "Optuna and automl," *Hyperparameter Optimization in Machine Learning: Make Your Machine Learning and Deep Learning Models More Efficient*, pp. 109–129, 2021.
- [25] J. Bezanson, J. Chen, B. Chung, S. Karpinski, V. B. Shah, J. Vitek, and L. Zoubitzky, "Julia: Dynamism and performance reconciled by design," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–23, 2018.